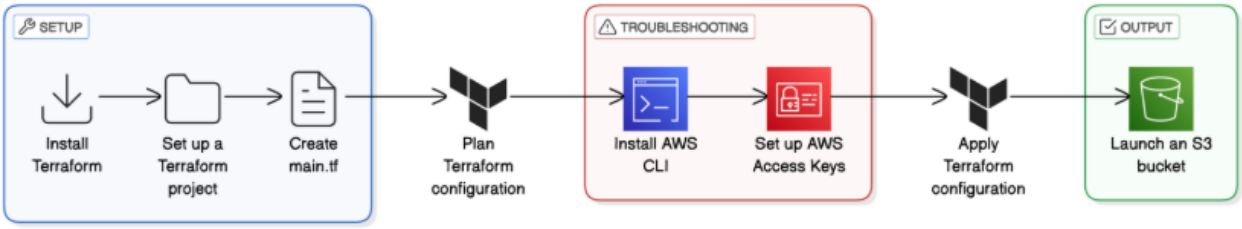
AWS Cloud Deployment with Terraform

In this project, I will use Terraform to **automate** the deployment of an S3 bucket and uploading files.

.



Download Terraform

1. The first step is to download the terraform.exe file

   Download the Terraform package for your operating system: https://developer.hashicorp.com/terraform/downloads

Set up your Terraform project

Now that Terraform is installed, let's create a project folder. This folder will store the files that tell Terraform what to build.

1. Create a project directory called: network_directory
2. Navigate your terminal to the nextwork_terraform directory.
3. Inside the nextwork_terraform directory, create a new file called: main.tf

```
C:\Users\russe\terraform_1.14.7_windows_amd64>terraform version
Terraform v1.14.7
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.14.8. You can update by downloading from https://developer.hashicorp.com/terraform/install

C:\Users\russe\terraform_1.14.7_windows_amd64>mkdir nextwork_terraform

C:\Users\russe\terraform_1.14.7_windows_amd64>cd nextwork_terraform

C:\Users\russe\terraform_1.14.7_windows_amd64\nextwork_terraform>type nul > main.tf

C:\Users\russe\terraform_1.14.7_windows_amd64\nextwork_terraform>
```

4. Verify creation by using cmd: dir
5. Edit the main.tf and add the script, adding your region, your own name and a really long random number

```
C:\Users\russe\terraform_1.14.7_windows_amd64\nextwork_terraform>dir
 Volume in drive C is WINDOWS
 Volume Serial Number is 7212-4675

 Directory of C:\Users\russe\terraform_1.14.7_windows_amd64\nextwork_terraform

04/08/2026  11:31 AM    <DIR>          .
04/08/2026  11:25 AM    <DIR>          ..
04/08/2026  11:31 AM                 0 main.tf
               1 File(s)              0 bytes
               2 Dir(s)  97,945,829,376 bytes free

C:\Users\russe\terraform_1.14.7_windows_amd64\nextwork_terraform>notepad main.tf
```

```terraform
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "nextwork-unique-bucket-russgeis-16228474722847482947 4" # Make sure this bucket name is globally unique by typing a long
random number
}

resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
  bucket = aws_s3_bucket.my_bucket.id

  block_public_acls       = true
  ignore_public_acls      = true
  block_public_policy     = true
  restrict_public_buckets = true
}
```

6. Use the Terraform registry's documentation on S3 buckets to add more features

Initialize Terraform Script

- Head back to your local terminal.
- **Initalize Terraform** by running the following command: terraform init

```
C:\Users\russe\terraform_1.14.8_windows_amd64>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.39.0...
- Installed hashicorp/aws v6.39.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the prov
ider
selections it made above. Include this file in your version control repo
sitory
so that Terraform can guarantee to make the same selections by default w
hen
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" t
o see
any changes that are required for your infrastructure. All Terraform com
mands
should now work.

If you ever set or change modules or backend configuration for Terraform

,
rerun this command to reinitialize your working directory. If you forget
, other
commands will detect it and remind you to do so if necessary.

C:\Users\russe\terraform_1.14.8_windows_amd64>
```

7. Next run: terraform plan
    a. This will show what will be created, changed, or deleted before you actually execute

```
C:\Users\russe\terraform_1.14.8_windows_amd64>terraform plan

Terraform used the selected providers to generate the following
execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

  # aws_s3_bucket.my_bucket will be created
  + resource "aws_s3_bucket" "my_bucket" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "nextwork-unique-bucket-russgeis-16228474722847482947 4"
      + bucket_domain_name          = (known after apply)
      + bucket_namespace            = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_region               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = "us-east-1"
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)
```

```
  # aws_s3_bucket_public_access_block.my_bucket_public_access_block will be created
  + resource "aws_s3_bucket_public_access_block" "my_bucket_public_access_block" {
      + block_public_acls       = true
      + block_public_policy     = true
      + bucket                  = (known after apply)
      + id                      = (known after apply)
      + ignore_public_acls      = true
      + region                  = "us-east-1"
      + restrict_public_buckets = true
    }

Plan: 2 to add, 0 to change, 0 to destroy.

──────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform
can't guarantee to take exactly these actions if you run "terraform
apply" now.

C:\Users\russe\terraform_1.14.8_windows_amd64>
```

Adding AWS CLI

In most cases, there will be some errors because of incorrect token or validation. This can be resolved by having Terraform authenticate with AWS and manage our resources.

1. Run this script in powershell to download AWS CLI
    a. Invoke-WebRequest -Uri "https://awscli.amazonaws.com/AWSCLIV2.msi" -OutFile "AWSCLIV2.msi"
    b. Start-Process msiexec.exe -ArgumentList "/i AWSCLIV2.msi" -Wait

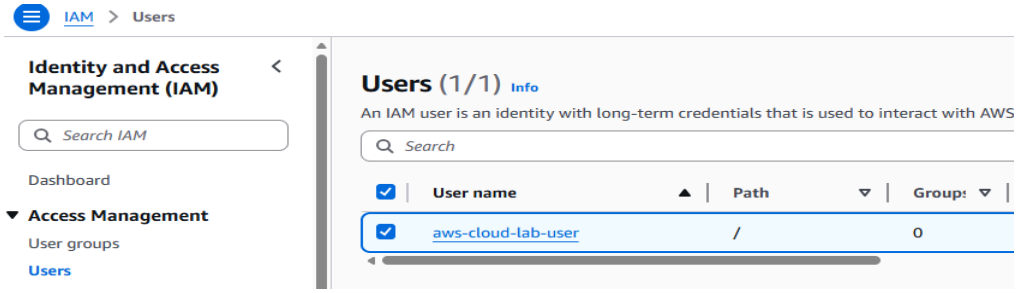
Verify CLI Installation

1. Once you've installed AWS CLI, open your terminal and run:

   Aws - -version
   aws configure

```
PS C:\Users\russe> Invoke-WebRequest -Uri "https://awscli.amazonaws.com/AWSCLIV2.msi" -OutFile "AWSCLIV2.msi"
PS C:\Users\russe> Start-Process msiexec.exe -ArgumentList "/i AWSCLIV2.msi" -Wait
PS C:\Users\russe> aws --version
aws-cli/2.34.27 Python/3.14.3 Windows/11 exe/AMD64
PS C:\Users\russe> aws configure
AWS Access Key ID [****************L4DM]:
```

2. Usually the access key would be blank
3. Go to AWS management console.
4. Go to **IAM** and click on **users** at the left and select your admin user



5. Once you select your admin account, click the **security credentials** tab
6. In the **Access keys** section, select **Create access key**.
7. Select **Command Line Interface (CLI)**
8. For the description add something like this:
    a. Created to get programmatic access to AWS. Using this key for the NextWork project on Terraform.

9. Next **Create Access Key**. BE SURE TO DOWNLOAD THE CSV

Configure the AWS CLI

- Head back to your local terminal again - this time you have credentials handy to access AWS CLI. Nice!
    - If your previous terminal sessions has closed, run aws configure again.
- Copy the **Access Key ID** and **Access Key** from the IAM console, and paste it in your terminal. Make sure to add the right region.
- Now running **terraform init** and **terraform plan** should give you all the right initializations

```
C:\Users\russe\terraform_1.14.8_windows_amd64>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.39.0...
- Installed hashicorp/aws v6.39.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the prov
ider
selections it made above. Include this file in your version control repo
sitory
so that Terraform can guarantee to make the same selections by default w
hen
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" t
o see
any changes that are required for your infrastructure. All Terraform com
mands
should now work.

If you ever set or change modules or backend configuration for Terraform
,
rerun this command to reinitialize your working directory. If you forget
, other
commands will detect it and remind you to do so if necessary.

C:\Users\russe\terraform_1.14.8_windows_amd64>
```

Applying Terraform configs and launching s3 bucket

Apply Terraform Configuration

1. Run the following command: **terraform apply**

2. Type **yes** and press **Enter** to apply the changes.

```
aws_s3_bucket.my_bucket: Creating...
aws_s3_bucket.my_bucket: Creation complete after 2s [id=nextwork-unique-bucket-russgeis-16228474722847482947 4]
aws_s3_bucket_public_access_block.my_bucket_public_access_block: Creating...
aws_s3_bucket_public_access_block.my_bucket_public_access_block: Creation complete after 0s [id=nextwork-unique-buc
ket-russgeis-16228474722847482947 4]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

C:\Users\russe\terraform_1.14.8_windows_amd64>
```

Verify S3 Bucket Creation

- Log in to the AWS Management Console as your IAM Admin user.
- Head to the S3 console.
- You should see your newly created S3 bucket in the console

○	nextwork-unique-bucket-russgeis-16228474722847482947 4	US East (N. Virginia) us-east-1	April 8, 2026, 16:40:29 (UTC-05:00)